final

December 11, 2025

1 initialize model

[55]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# import lightgbm as lgb


from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)
```

[56]:
```
df = pd.read_csv('data.csv')
df.rename(columns={ df.columns[0]: "date" }, inplace = True)
```

2 XGBoost

[ ]:
```
base_features = [
    '^VIX',
    '^GSPC',
    'CCC_10Y_Spread',
    'Unemployment',
    'Inflation',
    'GDP_Growth',
    'Corporate_Leverage',
    'YieldCurveSlope',
    'IndPro_Growth',
    'CP_Growth',
]
```

```python
# Assume df is already loaded externally
df = df.sort_values("date")

# Convenience alias
df["spread"] = df["CCC_10Y_Spread"]

# Directional features on spread (using levels)
df["spread_ret_30"] = df["spread"].diff(30)
df["spread_ret_60"] = df["spread"].diff(60)
df["spread_ret_180"] = df["spread"].diff(180)
df["spread_ret_360"] = df["spread"].diff(365)

df["spread_vol_30"] = df["spread"].diff().rolling(30).std()
df["spread_vol_60"] = df["spread"].diff().rolling(60).std()
df["spread_vol_180"] = df["spread"].diff().rolling(180).std()
df["spread_vol_360"] = df["spread"].diff().rolling(365).std()

# First-difference (change) of base macro features
for col in base_features:
    df[col + "_week"] = df[col].diff(7)
    df[col + "_month"] = df[col].diff(30)
    df[col + "_half_y"] = df[col].diff(180)
    df[col + "_year"] = df[col].diff(365)

# Drop rows with NaNs from shifting/diff/rolling
df = df.dropna().copy()


train_end = '2016-12-31'
val_end   = '2021-12-31'

train = df[df['date'] <= train_end]
val   = df[(df['date'] > train_end) & (df['date'] <= val_end)]
test  = df[df['date'] > val_end]



# Numeric features: original + engineered
feature_cols_num = (
    base_features +
    [ "spread_ret_30", "spread_ret_60",
        "spread_ret_180", "spread_ret_360",
        "spread_vol_30", "spread_vol_60",
        "spread_vol_180", "spread_vol_360",]
    + [col + "_week" for col in base_features] +
    [col + "_month" for col in base_features] +
    [col + "_half_y" for col in base_features] +
```

```
    [col + "_year" for col in base_features]
)

# If you later add categoricals, put names here
feature_cols_cat = []  # e.g. ['rating', 'sector']

X_train = train[feature_cols_num]
X_val   = val[feature_cols_num]
X_test  = test[feature_cols_num]

X_train_lgb = X_train
X_val_lgb   = X_val
X_test_lgb  = X_test


# Target is the spread LEVEL
y_train = train["spread"]
y_val   = val["spread"]
y_test  = test["spread"]



preprocess = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), feature_cols_num),
        ('cat', OneHotEncoder(handle_unknown='ignore'), feature_cols_cat),
    ],
    remainder='drop'
)

preprocess.fit(X_train)

X_train_t = preprocess.transform(X_train)
X_val_t   = preprocess.transform(X_val)
X_test_t  = preprocess.transform(X_test)
```

[ ]:
```
# xgb_model = XGBRegressor(
#     n_estimators=3000,
#     max_depth=20,
#     learning_rate=0.01,
#     subsample=0.8,
#     colsample_bytree=0.8,
#     min_child_weight=5,
#     reg_lambda=2.0,
#     reg_alpha=1.0,
#     objective='reg:squarederror',
#     random_state=42,
```

```
# )

# xgb_model.fit(
#     X_train_t, y_train,
#     eval_set=[(X_val_t, y_val)],
#     verbose=False
# )
```

[ ]:
```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=0.8, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.01, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=20,
             max_leaves=None, min_child_weight=5, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=3000,
             n_jobs=None, num_parallel_tree=None, …)
```

[ ]:
```
# lgbm_model = LGBMRegressor(
#     n_estimators=3000,
#     max_depth=-1,           # let num_leaves control tree size
#     learning_rate=0.01,
#     num_leaves=31,
#     subsample=0.8,
#     colsample_bytree=0.8,
#     objective='regression',
#     random_state=42,
#     # if your version accepts n_jobs, you can add: n_jobs=-1
# )

# lgbm_model.fit(
#     X_train_lgb, y_train,
#     eval_set=[(X_val_lgb, y_val)],
#     # eval_metric='rmse',
# )
```

```
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of
testing was 0.000647 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 13488
[LightGBM] [Info] Number of data points in the train set: 6941, number of used
features: 53
[LightGBM] [Info] Start training from score 0.114654
```

[ ]:
```
LGBMRegressor(colsample_bytree=0.8, learning_rate=0.01, n_estimators=3000,
              objective='regression', random_state=42, subsample=0.8)
```

3 Metrics

3.1 XGBoost

[60]:
```
def metrics_with_direction(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)

    # MAPE on levels
    denom = np.where(y_true == 0, 1e-8, y_true)
    mape = np.mean(np.abs((y_true - y_pred) / denom)) * 100

    # Directional Accuracy based on level changes
    true_dir = np.sign(y_true[1:] - y_true[:-1])
    pred_dir = np.sign(y_pred[1:] - y_pred[:-1])
    da = np.mean(true_dir == pred_dir)

    return mae, rmse, r2, mape, da
```

[61]:
```
y_pred_train_x = xgb_model.predict(X_train_t)
y_pred_val_x   = xgb_model.predict(X_val_t)

train_mae, train_rmse, train_r2, train_mape, train_da = metrics_with_direction(
    y_train.values, y_pred_train_x
)
val_mae, val_rmse, val_r2, val_mape, val_da = metrics_with_direction(
    y_val.values, y_pred_val_x
)

print("=== TRAIN METRICS (level) ===")
print(f"MAE :  {train_mae:.4f}")
print(f"RMSE: {train_rmse:.6f}")
print(f"R²  : {train_r2:.4f}")
print(f"MAPE: {train_mape:.2f}%")
print(f"DA  : {train_da:.4f}")

print("\n=== VALIDATION METRICS (level) ===")
print(f"MAE :  {val_mae:.4f}")
print(f"RMSE: {val_rmse:.6f}")
print(f"R²  : {val_r2:.4f}")
print(f"MAPE: {val_mape:.2f}%")
print(f"DA  : {val_da:.4f}")
```

```
=== TRAIN METRICS (level) ===
```

```
MAE :  0.0016
RMSE: 0.003258
R²  : 0.9967
MAPE: 1.62%
DA  : 0.6524

=== VALIDATION METRICS (level) ===
MAE :  0.0145
RMSE: 0.019966
R²  : 0.4211
MAPE: 19.51%
DA  : 0.5912
```

[62]:
```python
y_pred_train = lgbm_model.predict(X_train)
y_pred_val   = lgbm_model.predict(X_val)

train_mae, train_rmse, train_r2, train_mape, train_da = metrics_with_direction(
    y_train.values, y_pred_train
)
val_mae, val_rmse, val_r2, val_mape, val_da = metrics_with_direction(
    y_val.values, y_pred_val
)

print("=== TRAIN METRICS (level) ===")
print(f"MAE :  {train_mae:.4f}")
print(f"RMSE: {train_rmse:.6f}")
print(f"R²  : {train_r2:.4f}")
print(f"MAPE: {train_mape:.2f}%")
print(f"DA  : {train_da:.4f}")

print("\n=== VALIDATION METRICS (level) ===")
print(f"MAE :  {val_mae:.4f}")
print(f"RMSE: {val_rmse:.6f}")
print(f"R²  : {val_r2:.4f}")
print(f"MAPE: {val_mape:.2f}%")
print(f"DA  : {val_da:.4f}")
```

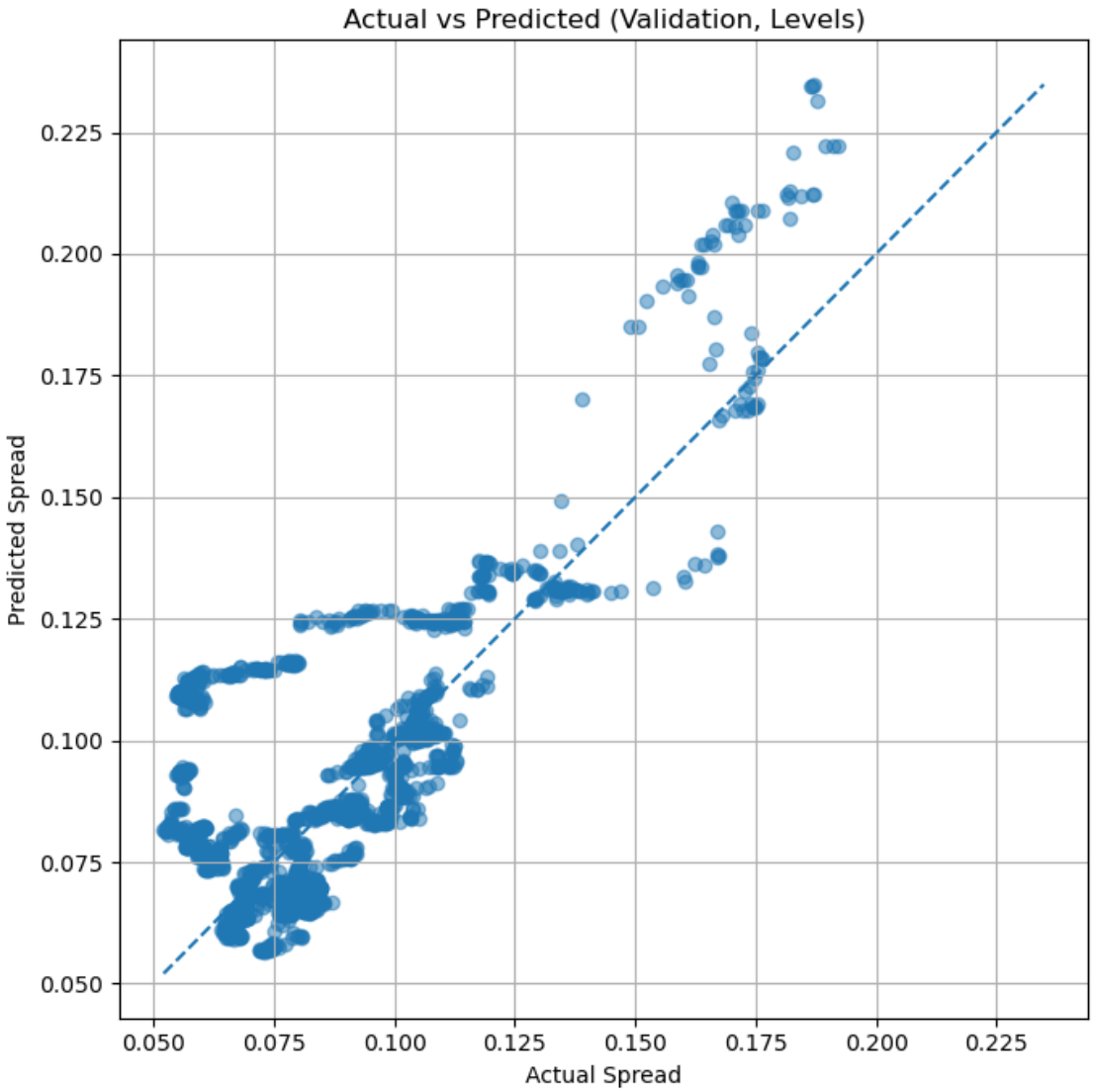
```
=== TRAIN METRICS (level) ===
MAE :  0.0004
RMSE: 0.000582
R²  : 0.9999
MAPE: 0.37%
DA  : 0.7561

=== VALIDATION METRICS (level) ===
MAE :  0.0154
RMSE: 0.021807
R²  : 0.3094
```

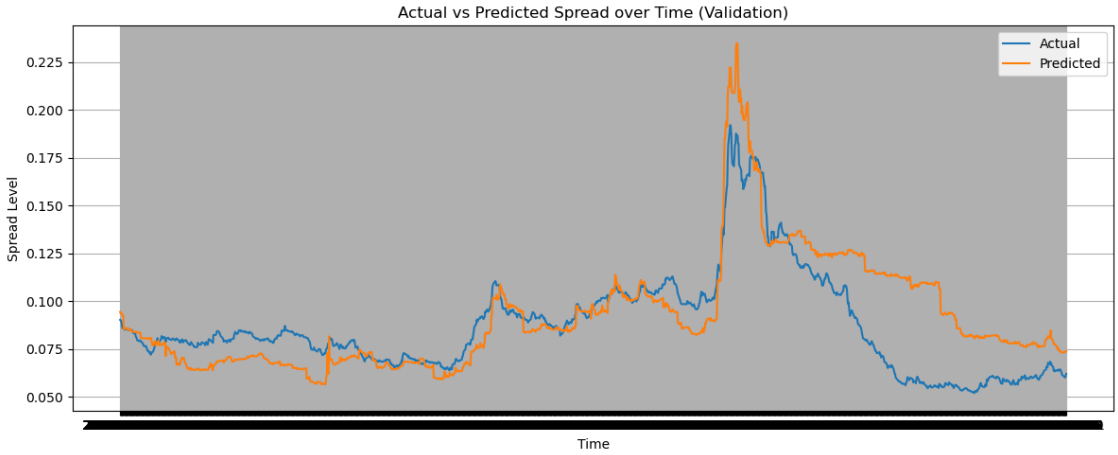
```
MAPE: 18.66%
DA   : 0.6652
```

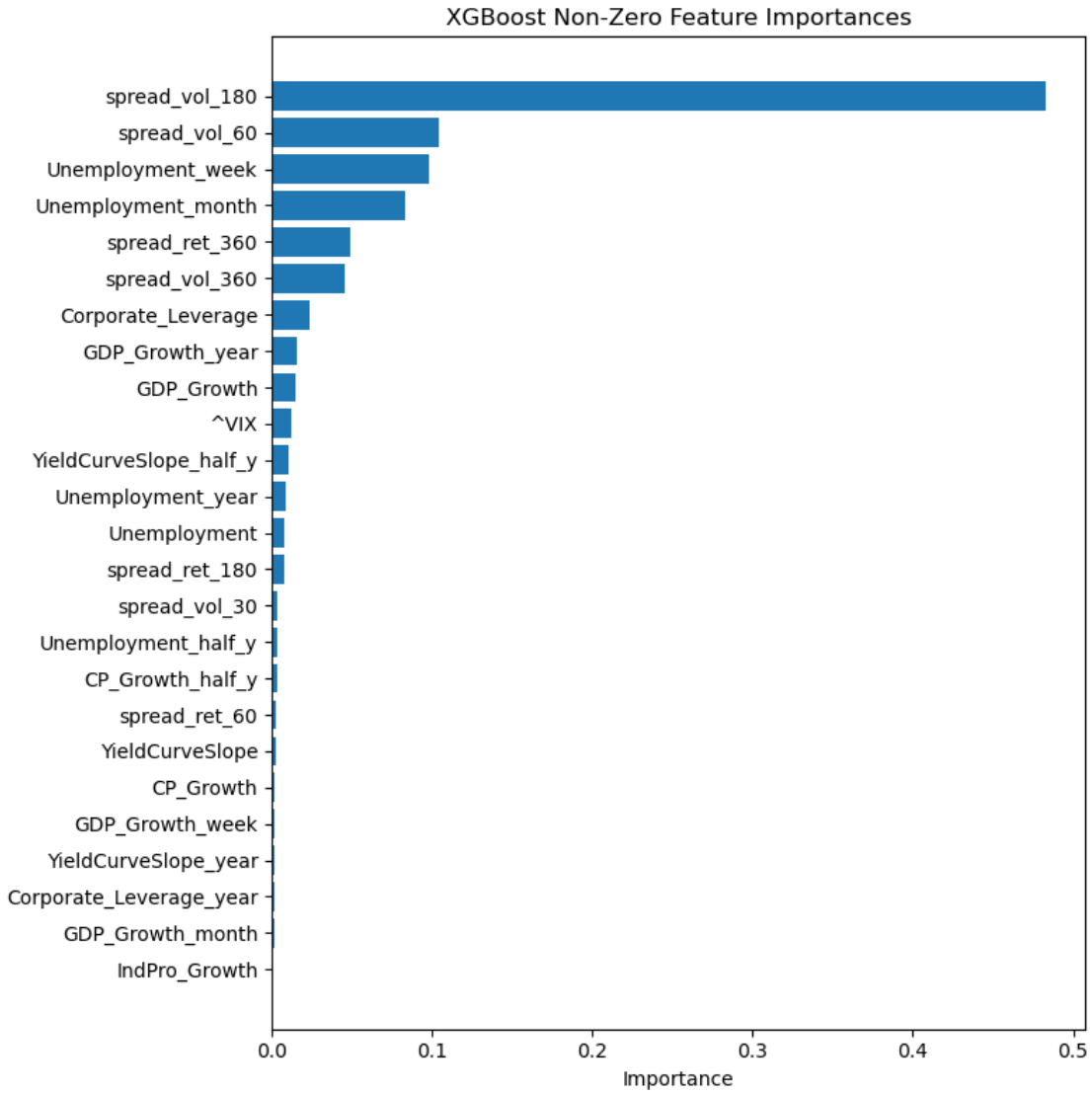
4 Plots

[63]:
```python
plt.figure(figsize=(7, 7))
plt.scatter(y_val, y_pred_val_x, alpha=0.5)

min_val = min(y_val.min(), y_pred_val_x.min())
max_val = max(y_val.max(), y_pred_val_x.max())
plt.plot([min_val, max_val], [min_val, max_val], linestyle='--')

plt.xlabel("Actual Spread")
plt.ylabel("Predicted Spread")
plt.title("Actual vs Predicted (Validation, Levels)")
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Time series: Actual vs Predicted over time ---
val_df = pd.DataFrame(
    {
        "Actual": y_val.values,
        "Predicted": y_pred_val_x,
    },
    index=val["date"]
)

plt.figure(figsize=(12, 5))
plt.plot(val_df.index, val_df["Actual"], label="Actual")
plt.plot(val_df.index, val_df["Predicted"], label="Predicted")
plt.xlabel("Time")
plt.ylabel("Spread Level")
plt.title("Actual vs Predicted Spread over Time (Validation)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Feature Importances ---
importances = xgb_model.feature_importances_

# importances = lgbm_model.feature_importances_

# Indices of non-zero importances
nz_idx = np.where(importances > 0.001)[0]
```

```
# Sorted indices (only non-zero)
sorted_nz_idx = nz_idx[np.argsort(importances[nz_idx])]

# Corresponding feature names
nz_features = np.array(feature_cols_num)[sorted_nz_idx]
nz_importances = importances[sorted_nz_idx]

plt.figure(figsize=(8, 8))
plt.barh(range(len(sorted_nz_idx)), nz_importances)
plt.yticks(range(len(sorted_nz_idx)), nz_features)
plt.xlabel("Importance")
plt.title("XGBoost Non-Zero Feature Importances")
plt.tight_layout()
plt.show()
```

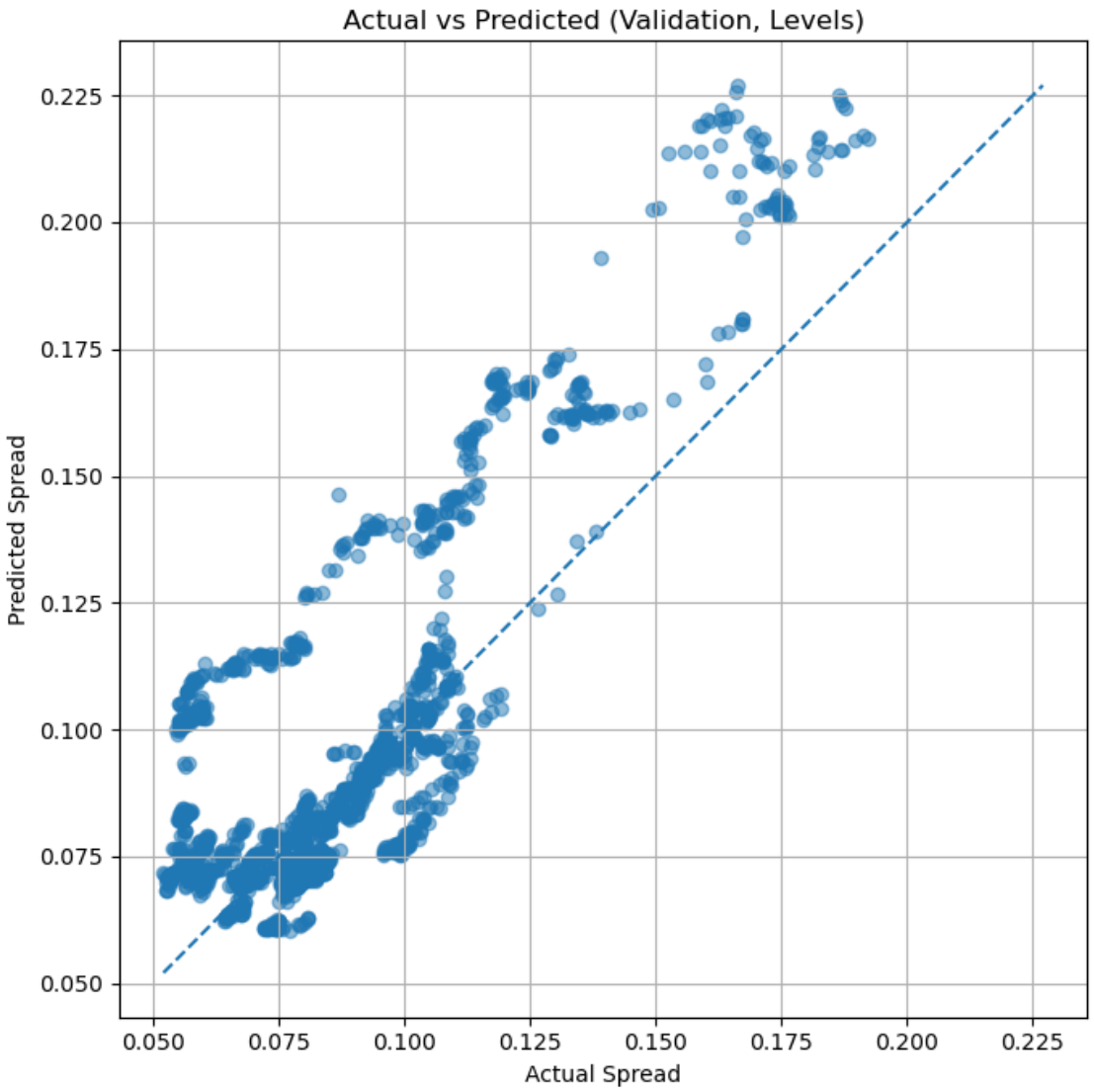


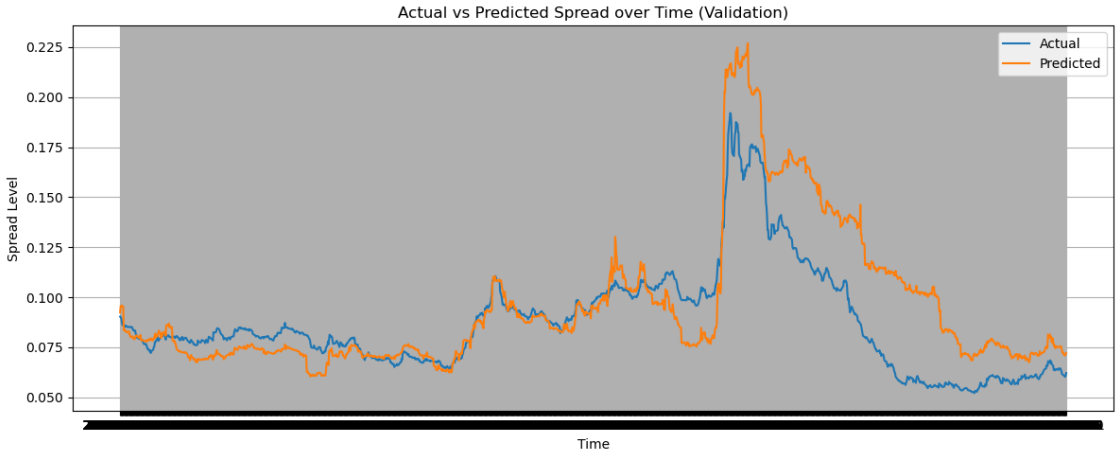

Actual vs Predicted Spread over Time (Validation)
Actual
Predicted
Spread Level
0.225
0.200
0.175
0.150
0.125
0.100
0.075
0.050
Time

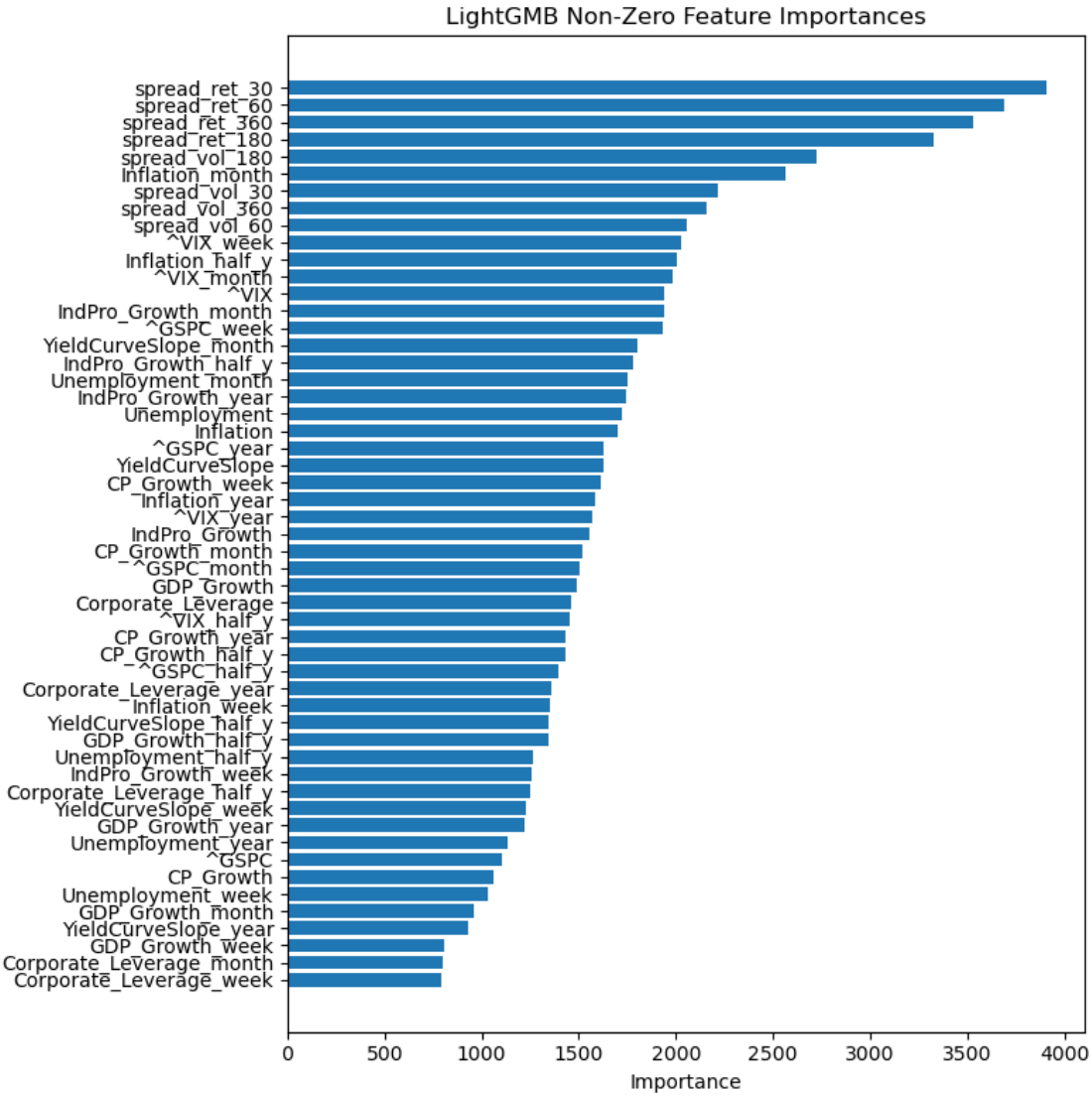


[64]:
```
plt.figure(figsize=(7, 7))
plt.scatter(y_val, y_pred_val, alpha=0.5)

min_val = min(y_val.min(), y_pred_val.min())
max_val = max(y_val.max(), y_pred_val.max())
plt.plot([min_val, max_val], [min_val, max_val], linestyle='--')

plt.xlabel("Actual Spread")
plt.ylabel("Predicted Spread")
plt.title("Actual vs Predicted (Validation, Levels)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# --- Time series: Actual vs Predicted over time ---
val_df = pd.DataFrame(
    {
        "Actual": y_val.values,
        "Predicted": y_pred_val,
    },
    index=val["date"]
)

plt.figure(figsize=(12, 5))
plt.plot(val_df.index, val_df["Actual"], label="Actual")
plt.plot(val_df.index, val_df["Predicted"], label="Predicted")
plt.xlabel("Time")
plt.ylabel("Spread Level")
plt.title("Actual vs Predicted Spread over Time (Validation)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Feature Importances ---
# importances = xgb_model.feature_importances_

importances = lgbm_model.feature_importances_

# Indices of non-zero importances
nz_idx = np.where(importances > 0.001)[0]

# Sorted indices (only non-zero)
sorted_nz_idx = nz_idx[np.argsort(importances[nz_idx])]

# Corresponding feature names
nz_features = np.array(feature_cols_num)[sorted_nz_idx]
nz_importances = importances[sorted_nz_idx]

plt.figure(figsize=(8, 8))
plt.barh(range(len(sorted_nz_idx)), nz_importances)
plt.yticks(range(len(sorted_nz_idx)), nz_features)
plt.xlabel("Importance")
plt.title("LightGMB Non-Zero Feature Importances")
plt.tight_layout()
plt.show()
```


Actual vs Predicted (Validation, Levels)
Predicted Spread
Actual Spread
0.050
0.075
0.100
0.125
0.150
0.175
0.200
0.225


Actual vs Predicted Spread over Time (Validation)
Spread Level
Time
Actual
Predicted
0.050
0.075
0.100
0.125
0.150
0.175
0.200
0.225


LightGMB Non-Zero Feature Importances
spread_ret_30
spread_ret_60
spread_ret_360
spread_ret_180
spread_vol_180
Inflation_month
spread_vol_30
spread_vol_360
spread_vol_60
^VIX_week
Inflation_half_y
^VIX_month
^VIX
IndPro_Growth_month
^GSPC_week
YieldCurveSlope_month
IndPro_Growth_half_y
Unemployment_month
IndPro_Growth_year
Unemployment
Inflation
^GSPC_year
YieldCurveSlope
CP_Growth_week
Inflation_year
^VIX_year
IndPro_Growth
CP_Growth_month
^GSPC_month
GDP_Growth
Corporate_Leverage
^VIX_half_y
CP_Growth_year
CP_Growth_half_y
^GSPC_half_y
Corporate_Leverage_year
Inflation_week
YieldCurveSlope_half_y
GDP_Growth_half_y
Unemployment_half_y
IndPro_Growth_week
Corporate_Leverage_half_y
YieldCurveSlope_week
GDP_Growth_year
Unemployment_year
^GSPC
CP_Growth
Unemployment_week
GDP_Growth_month
YieldCurveSlope_year
GDP_Growth_week
Corporate_Leverage_month
Corporate_Leverage_week
0
500
1000
1500
2000
2500
3000
3500
4000
Importance